IrisPathQ: Quantum-Enhanced Flight Route Optimization Using QAOA

Patel, Tisha[1]; Sousa, Paulo[1]

[1] Algonquin College { pate1416@algonquinlive.com, sousap@algonquincollege.com }

ABSTRACT

This paper presents IrisPathQ[1], a hybrid quantum-classical framework that integrates some heuristics with the quantum approach, based on QAOA and QUBO, that can outperform greedy approaches when conflict penalties are introduced, achieving significant reductions in total operational cost. Results highlight the potential of quantum computing in aviation logistics, particularly in scenarios with interdependent flight routes.

1. INTRODUCTION

Air Traffic Control (ATC) is fundamental for our daily life activities in society and has huge impact when problems can happen due to scheduling or safety in the shared airspace. Real-world planning and scheduling problems typically deal with heuristics to make the solution feasible for complex problems, especially when the number of instances increases. The Tail Assignment Problem (TAP) and Traveling Salesman Problem (TSP) are classical NP-hard problems that can be applied to flight routing [Lucas, A. (2014), Fuertes, M. et al (2020).]. However, TSP is about finding the shortest possible route for a salesman visiting a set of cities, TAP is about assigning airplanes (identified by their tail numbers) to scheduled flights in the most efficient way, while the. Both are combinatorial optimization problems but TAP deals with resources (aircraft) and constraints like maintenance and regulations as well as fuel efficiency, weather avoidance, and airspace conflict resolution. This problem is especially important in the context of energy optimization and climate change scenario and several recent researches are focusing on aviation optimization problems [Stollenwerk, T., et al. (2019)., Fuertes, M., et al*(2021)*, Blanco, M. et al. (2022), Adamouization (2023), Nan Xu et al (2024), Ye, M. et al (2024)].

One possible approach is to use classical greedy algorithms that provide fast solutions but often fail to capture global interdependencies between flights. Airline operations require efficient scheduling and routing to minimize fuel consumption, reduce delays, and avoid hazardous weather. Traditional optimization methods, such as greedy heuristics or more advanced methods such as Mixed Integer Linear Programming (MILP), often struggle with higher scalability and interdependencies between flights.

TAP involves assigning aircraft to flights under operational constraints, while TSP seeks the shortest route visiting all cities. Both can be reformulated using quantum approach version suitable for quantum optimization. In this work, we investigate what paradigms for solving such problems, such as QAOA (Quantum Approximate Optimization Algorithm) [Blekos, K., et al. (2023), Combarro, E. F., & González-Castillo, S. (2023)] and QUBO (Quadratic Unconstrained Binary Optimization) can offer when quantum computing is applied to explore solution spaces globally.

2. PROBLEM DESCRIPTION

Airlines face daily challenges in solving complex, large-scale planning problems that involve multiple resource types, including passengers, crew, aircraft, maintenance, and ground staff. The airline planning process is typically sequential, beginning with timetable construction and followed by aircraft and crew scheduling steps, each of which constitutes a large-scale optimization problem with distinct objectives but a common overarching goal: maximizing profitability, safety, and crew satisfaction while minimizing disruptions. These planning tasks must simultaneously satisfy a wide range of regulatory, operational, and quality constraints, making them computationally demanding.

One critical subproblem in fleet planning is the Tail-Assignment Problem (TAP) [Fuentes et al, 2020], where the objective is to determine which individual aircraft (identified by their tail numbers) should operate each

[1] Repository available at https://github.com/tishap27/IrisPathQ/.

scheduled flight. A sequence of flights operated consecutively by the same aircraft is referred to as a route.

For a route to be legally feasible, it must satisfy several operational constraints. For instance, the buffer time between the arrival of one flight and the departure of the next, known as the minimum turn time, must exceed a threshold that depends on factors such as flight type (domestic versus international), airport infrastructure, time of day, and aircraft-specific characteristics. Additional constraints include destination restrictions, which prevent certain aircraft from operating at specific airports due to limitations such as insufficient runway length or engine thrust capacity. Curfew restrictions further limit operations by prohibiting noisy aircraft from flying during night hours at airports located near urban centers. Finally, both short- and long-term maintenance constraints require aircraft to regularly visit airports equipped with maintenance facilities, ensuring compliance with safety and operational standards.

2.1. Modelling

Airline route optimization can be formulated as a combinatorial problem in which a set of $\boldsymbol{N}$ flights must each be assigned one of $\boldsymbol{M}$ possible routes. The objective is to minimize total fuel consumption while simultaneously satisfying operational constraints such as storm detours (which impose additional fuel penalties), airspace conflicts (arising when multiple flights attempt to traverse the same waypoint at the same time), and aircraft-specific limitations (e.g., runway length or thrust restrictions).

For example, consider a simplified case with two flights (F_0, F_1) and two possible routes (R_0, R_1). The fuel costs are given by the matrix:

	R_0	R_1
F_0	4	6
F_1	5	7

Table 1: Flight/route simple example

If both flights select the same route, a conflict penalty should be applied (for instance, incurring in the value 10). The greedy algorithm would assign $F_0 \to R_0$ and $F_1 \to R_0$, yielding a total cost of $4 + 5 + 10 = 19$.

2.3. Basic approaches

Being a real time problem, one can basically consider two different approaches based on classical and quantum computation, as mentioned as follows: the greedy heuristic approach and the application of quantum optimization.

A. Greedy heuristic approach

A classical greedy approach provides computational efficiency by selecting, for each flight, the locally cheapest route in terms of fuel cost. However, this sequential decision-making process neglects interdependencies between flights. When conflicts occur—such as multiple aircraft selecting overlapping routes—the greedy method can yield suboptimal global solutions.

Note that the greedy algorithm provides a baseline solution by independently selecting, for each flight, the route with the lowest immediate fuel cost. This approach operates sequentially and does not account for interdependencies between flights.

Formally, for flight $f \in F$ with route set R_f, the greedy choice is:

$$r_f^* = \arg\min_{r \in R_f} \text{fuel}(r)$$

In the illustrative example with two flights (F_0, F_1) and two candidate routes (R_0, R_1), the greedy algorithm proceeds as follows:

- Flight F_0 selects R_0 with cost 4, since $4 < 6$.
- Flight F_1 selects R_0 with cost 5, since $5 < 7$.

However, both flights choosing the same route incurs a **conflict penalty** of 10, applied after the selections are made. The total cost is therefore:

$$C_{\text{greedy}} = 4 + 5 + 10 = 19$$

This simple example demonstrates the limitation of the greedy approach: it optimizes locally without backtracking or global consideration, leading to suboptimal solutions when conflicts arise.

B. Quantum optimization (QAOA approach)

Quantum optimization methods, particularly QAOA, exploit quantum superposition to evaluate all possible route combinations simultaneously. Interesting cases used in aviation are now being explored and online resources can show how this approach can be applied in aviation [NordIQuEst, 2025] This global exploration enables the algorithm to incorporate conflict penalties directly into the cost function, thereby avoiding locally optimal but globally inefficient solutions.

This strategy incorporates the penalty into the optimization landscape and identifies the globally optimal assignment. The principle of this heuristic is based on approximation of binary string that can match with an

optimal expected value of a function and can use advantages of quantum computation [Farhi et al, 2014].

QAOA addresses the shortcomings of the greedy heuristic by incorporating conflict penalties directly into the optimization landscape. The problem is encoded in a Quadratic Unconstrained Binary Optimization (QUBO) formulation [Kochenberger, G, 2024.], where both fuel costs and penalties are represented in the cost Hamiltonian.

The quantum circuit initializes all possible route combinations in superposition. Through parameterized cost and mixer Hamiltonians, QAOA explores the solution space globally, assigning higher energy to conflicting choices and thereby reducing their probability of measurement.

In the same two-flight example:

- Flight F_0 selects R_1 with cost 6.
- Flight F_1 selects R_0 with cost 5.

Because the conflict penalty is embedded in the cost function, the quantum optimizer avoids the overlapping assignment. The total cost is:

$$C_{\text{QAOA}} = 6 + 5 = 11$$

This solution is globally optimal under the given constraints, outperforming the greedy heuristic. Importantly, QAOA evaluates the same candidate routes as the classical method but explores alternative combinations to identify lower-cost solutions when conflicts are present.

2.3. Comparative Analysis

The classical greedy heuristic and the quantum optimization approach differ fundamentally in their treatment of interdependencies between flights. The greedy method operates sequentially, selecting for each flight the route with the lowest immediate fuel cost. This approach is computationally efficient, with linear complexity in the number of flights, and is well-suited for large-scale problems where speed is prioritized. However, its local decision-making framework neglects global constraints such as airspace conflicts or maintenance requirements. As a result, the greedy algorithm often produces solutions that appear optimal at the individual flight level but are suboptimal when evaluated across the entire system.

In contrast, the quantum optimization approach, exemplified by QAOA, incorporates conflict penalties directly into the optimization landscape. By encoding both fuel costs and penalties into a unified cost Hamiltonian, QAOA evaluates all possible route combinations simultaneously. This global exploration enables the algorithm to avoid locally optimal but globally inefficient solutions. Although quantum optimization is computationally more demanding in terms of circuit depth and parameter tuning, it provides a mechanism to achieve solutions that minimize total fuel consumption while respecting operational constraints. The comparative analysis thus highlights a trade-off: the greedy heuristic offers speed and simplicity, whereas QAOA provides robustness and global optimality in complex, interdependent routing scenarios.

This comparison highlights the potential of quantum optimization to provide significant improvements over classical heuristics in aviation routing problems, particularly when multiple flights interact through shared constraints.

3. Route Optimization and Quantum Approximate Optimization Algorithm (QAOA)

3.1. Overview

This phase establishes a baseline for airline route optimization by comparing a classical greedy heuristic with the Quantum Approximate Optimization Algorithm (QAOA). The primary objective is to validate the quantum circuit design and to establish performance metrics against a classical benchmark. Route optimization in aviation is a critical problem, as it directly impacts fuel efficiency, operational costs, and environmental sustainability. By formulating the problem in both classical and quantum paradigms, this phase provides a foundation for subsequent experiments involving interdependent flight routes and conflict penalties.

3.2. Greedy Route Optimization

The baseline system employs a greedy heuristic, which independently selects the lowest-fuel route for each flight. This approach is computationally efficient, with linear complexity $O(n)$, but it neglects interdependencies between flights.

A. Methodology

Route Generation

Routes are generated using a preprocessing module implemented in C. Direct paths between origin and destination airports are computed using the **Haversine formula**, which calculates great-circle distances. For each flight, five alternative routes are produced, with

incremental fuel costs (+2% per alternative). This provides a simple yet scalable representation of route variability.

Example:

- Flight AC101 (YYZ → YVR)
  - Route 0: 2,500 nm → 10,036 kg fuel (baseline)
  - Route 1: 2,550 nm → 10,237 kg fuel (+2%)
  - Route 2: 2,601 nm → 10,437 kg fuel (+4%)
  - Route 3: 2,652 nm → 10,638 kg fuel (+6%)
  - Route 4: 2,703 nm → 10,839 kg fuel (+8%)

B. Classical Solver

The greedy solver selects the minimum-fuel route for each flight independently:

$$r_f^* = \arg\min_{r \in R_f} c_{fr}$$

where c_{fr} denotes the fuel cost of assigning flight f to route r.

This algorithm scales linearly with the number of flights but does not account for route conflicts, weather hazards, or maintenance constraints.

3.3. Quantum Solver: QAOA

The quantum solver employs the **Quantum Approximate Optimization Algorithm (QAOA)**, implemented using Qiskit Aer. Unlike the greedy heuristic, QAOA explores all route combinations simultaneously by encoding the optimization problem into a cost Hamiltonian.

A. Circuit Architecture

- **Qubits:** 10 (2 qubits per flight).
- **Gate Functions:**
  - **Hadamard (H):** Initializes superposition, enabling exploration of all routes.
  - **RZ(γ):** Encodes cost Hamiltonian, marking low-fuel solutions.
  - **RX(β):** Mixer Hamiltonian, enabling quantum tunneling between states.
  - **Measurement (M):** Collapses quantum states to classical route assignments.

B. Parameters

- $\gamma = 1.0$: Cost encoding strength.
- $\beta = 0.5$: Exploration strength.
- Circuit depth: 4 layers, 35 gates.
- Simulation: 500 measurement shots.

This configuration validates the QAOA circuit design and demonstrates its ability to recover global optima in simple, conflict-free scenarios.

C. Results

Test Case: 5 Flights, 25 Route Options

Flight	Origin → Destination	Selected Route	Fuel (kg)
AC101	YYZ → YVR	Route 0	10,036
WS202	YUL → YYC	Route 5	8,655
AC303	YEG → YOW	Route 10	8,547
WS404	YWG → YHZ	Route 15	7,729
AC505	YYJ → YYT	Route 20	15,151

Table 2: Flight/route example 2

3.4. Analysis

Using the previous scenario, we could note these outcomes in the total fuel comsumption:

- Classical (Greedy): 50,119 kg
- Quantum (QAOA): 50,119 kg

It means there is no improvement and, in fact both approaches converge to identical solutions because:

- The cost matrix contains only diagonal entries (fuel costs), with no off-diagonal conflict penalties.
- The problem decouples, allowing each flight to be optimized independently.

The quantum circuit successfully identifies the global optimum, confirming the correctness of the QAOA formulation in this baseline case.

3.5. Limitations of the Greedy Approach

While computationally efficient, the greedy heuristic exhibits several limitations:

- Routes are restricted to direct origin–destination paths, missing potentially more fuel-efficient intermediate waypoints.
- The algorithm does not account for dynamic weather conditions such as thunderstorms or turbulence.
- Airspace congestion and shared waypoint conflicts are not considered, leading to unrealistic solutions in practice.
- Local optimization fails when flights interact, whereas quantum optimization can exploit global exploration to avoid conflicts.

This result demonstrates that QAOA can replicate classical greedy solutions in conflict-free scenarios, validating the quantum circuit design. Although no quantum advantage

is observed in this baseline case, the limitations of the greedy heuristic highlight the need for more sophisticated models. Future phases will incorporate waypoint networks, weather hazards, and conflict penalties to demonstrate the potential of quantum optimization in realistic airline routing problems.

4. IRISPATHQ PHASE 2: A* PATHFINDING IMPLEMENTATION

4.1. Overview

The **A*** (A-star) pathfinding algorithm is a graph traversal method widely used in navigation, robotics, and aviation [Nan Xu et al (2024)]. It finds the shortest path between a start and goal node by combining the actual cost to reach a node (g) with a heuristic estimate of the remaining cost (h), producing an evaluation function $f(n) = g(n) + h(n)$. This makes A* both **complete** (it always finds a solution if one exists) and **optimal** (it finds the shortest path if the heuristic is admissible).

A* pathfinding algorithm aims to generate realistic flight routes through waypoint networks. Unlike the direct origin–destination routing of Phase 1, this implementation accounts for **weather hazards**, **airspace conflicts**, and **multiple route alternatives**. The goal is to provide a richer optimization landscape for quantum algorithms such as QAOA.

In this approach, one can find a configurable number of route options per flight with cost variations introduced for quantum optimization comparison. In this case, conflict detection between routes are based on shared waypoints.

4.2. Basic algorithm

1. **Initialization:** Origin waypoint assigned with
$$g_{cost} = 0\,.$$
2. **Priority Queue:** Nodes sorted by
$$f_{cost} = g_{cost} + h_{cost}\,.$$
3. **Expansion:** Neighboring waypoints within a specific restriction (ex: 3000 nm) are explored.
4. **Penalty Assignment:** Hazardous waypoints incur doubled costs.
5. **Goal Check:** Search terminates when destination waypoint is reached.
6. **Backtracking:** Path reconstructed using parent pointers.

The **time complexity** of A* depends on the heuristic quality. In the worst case, with poor heuristics, A* expands nodes exponentially in the depth of the solution:

$$O(b^d)$$

where $b$is the branching factor (average number of neighbors per waypoint) and $d$is the depth of the shortest path.

The **space complexity** is also exponential, as A* stores all generated nodes in memory. For aviation routing, this is mitigated by limiting neighbor exploration (e.g., within 3000 nm) and applying admissible heuristics such as great-circle distance.

4.3. Example

For flight AC101 (YYZ → YVR), five routes were generated:

- Route 0: Distance = 1806.46 nm, Fuel = 10,035.91 kg, Time = 4.01 hrs
- Route 1: Distance = 1806.46 nm, Fuel = 10,236.63 kg, Time = 4.09 hrs
- Route 2: Distance = 1806.46 nm, Fuel = 10,437.00 kg, Time = 4.18 hrs

The resulting cost matrix (25×25) encodes both fuel costs and conflict penalties, exported to proper files. The advantages about this approach can be summarized by basically these points: we have introduced the hazard penalties, shared waypoints and bringing a more realistic path variation that can deal with conflict penalties. However, there are some limitations, for example the limit of neighborhood exploration to be accepted in the algorithm due to the acceptable complexity to reduce branching factor and improve realism.

<u>Improvements Over greedy</u>

Feature	Greedy	A*
Route Type	Direct origin–destination	Waypoint network navigation
Weather	Ignored	Hazard penalties
Conflicts	No detection	Shared waypoint penalties
Alternatives	Fixed +2% increments	Realistic path variations
Quantum Advantage	None	Enabled (conflict penalties)

Table 3: Greedy/A* comparison

5. REALISTIC APPROACH: CONFLICT INJECTION

5.1. Overview

The previous scenarios have in common the execution over independence instances. For more realistic scenarios, it is necessary to introduce **strategic conflict injection** into the airline route optimization problem to demonstrate the potential quantum advantage of the Quantum Approximate Optimization Algorithm (QAOA). By deliberately creating interdependencies between flight routes, the optimization landscape becomes non-trivial for classical greedy heuristics, which fail to account for global constraints. QAOA, by contrast, explores all route combinations simultaneously and identifies globally optimal solutions that avoid conflicts.

Conflicts were introduced deterministically by forcing **Route 0** of flight $N$to share waypoints with **Route 0** of flight $N+1$. This mechanism ensured that if the greedy solver selected Route 0 for all flights, multiple overlaps would occur, triggering penalties.

The conflict injection was implemented was included in the routing module to simulate this. The algorithm modifies route definitions such that the second-to-last waypoint of flight $N$is inserted into the second position of flight $N+1$. This creates a chain of overlapping waypoints across consecutive flights.

A. Conflict chain example:

- Flight 0 Route 0: YYZ → [CYOW] → YVR (conflicts with Flight 1)
- Flight 1 Route 0: YUL → [CYOW] → YYC (conflicts with Flight 2)
- Flight 2 Route 0: YEG → [CYYC] → YOW (conflicts with Flight 3)

B. Cost Matrix Construction

The optimization problem must be encoded into a cost matrix. For instance, suppose a square matrix of dimension 25×25, representing five routes per flight across five flights. One single example can have the following with some properties like:

- **Diagonal entries:** Fuel costs (7,729–16,363 kg).
- **Off-diagonal entries:** Conflict penalties of 10,000 kg for overlapping routes.
- **Total conflicts:** 52 conflict pairs.
- **Penalty pool:** 520,000 kg if all conflicts are triggered.

The conflict detection algorithm ensures symmetry in the matrix:

$$Q[i,j] = Q[j,i] = P$$

"if routes " i,j" share a waypoint".

5.2. Solver Mechanism

A. Classical Greedy Solver

The greedy heuristic selected Route 0 for each flight, minimizing individual fuel costs but ignoring conflicts. This resulted in 10 overlapping conflicts and a total cost of 150,119 kg.

Flight	Route	Fuel (kg)	Conflict
AC101	0	10,036	✗ (with WS202)
WS202	0	8,655	✗ (with AC303)
AC303	0	8,547	✗ (with WS404)
WS404	0	7,729	✗ (with AC505)
AC505	0	15,151	✗ (multiple)
Total		**150,119 kg**	**10 conflicts**

Table 4: Examples of conflicts

B. Quantum QAOA Solver

QAOA explored all $5^5 = 3{,}125$route combinations simultaneously. By embedding penalties into the cost Hamiltonian, the quantum solver avoided overlapping routes and achieved a conflict-free solution with a total cost of 50,119 kg.

Flight	Route	Fuel (kg)	Conflict
AC101	0	10,036	✓ None
WS202	0	8,655	✓ None
AC303	0	8,547	✓ None
WS404	0	7,729	✓ None
AC505	0	15,151	✓ None
Total		**50,119 kg**	**0 conflicts**

Table 4: Solver example

5.3. Quantum vs Classical Comparison

The comparative analysis is summarized in the following table.

Metric	Greedy	QAOA
Approach	Local optimization	Global exploration
Complexity	$O(n)$	$O(2^n)$
Fuel Cost	50,119 kg	50,119 kg
Penalties	100,000 kg	0 kg
Total Cost	150,119 kg	50,119 kg
Runtime	<1 sec	~0.2 sec
Optimality	Local minimum	Global minimum

Table 5: Quantum x classical solver comparison

The comparative results are summarized in Table 1. While both solvers achieve identical baseline fuel costs (50,119 kg), the greedy approach incurs 100,000 kg in penalties due to conflicts, whereas QAOA avoids all penalties. This

corresponds to a **66.6% reduction in total cost**, highlighting the quantum solver's ability to escape local minima.

Solver	Fuel (kg)	Penalties (kg)	Total (kg)
Classical (Greedy)	50,119	100,000	150,119
Quantum (QAOA)	50,119	0	50,119

Table 6: Performance comparison between classical greedy and quantum QAOA solvers

The experiment demonstrates that QAOA can outperform classical greedy heuristics when conflicts are present. By encoding penalties directly into the optimization landscape, QAOA avoids infeasible solutions that greedy algorithms cannot detect. This aligns with prior research on quantum optimization applied to airline scheduling and gate assignment problems【Rosenberg et al., 2016】【Stollenwerk et al., 2019】.

The results also highlight the trade-off between computational complexity and solution quality. While greedy algorithms scale linearly, they are prone to local minima. QAOA requires exponential state exploration but leverages quantum parallelism to achieve globally optimal solutions in interdependent routing scenarios.

5.4. Classical baselines and their limitations

Greedy selection minimizes per-flight route cost independently and assigns $x_{i,m} = 1$for the cheapest m per i. Its appeal is computational efficiency, but the method does not anticipate cross-flight conflicts, so penalties enter after independent choices are made. Weather-aware A* improves local route scoring by integrating detours and weather multipliers into the route cost, but A* remains a per-flight optimizer. In scenarios where two locally cheap routes share a waypoint, both Greedy and A* converge to the same globally suboptimal selection because they neglect the pairwise coupling terms in Equation (6). This illustrates a general limitation of local methods in globally coupled routing tasks.

6. Mixed Integer Linear Programming (MILP) Formulation

Mixed Integer Linear Programming (MILP) is a mathematical optimization framework that combines continuous and discrete decision variables, making it a powerful tool for modeling complex real-world problems such as scheduling, logistics, and resource allocation [Nemhauser, G. L., & Wolsey, 1988].

6.1. Overview

A Mixed Integer Linear Program (MILP) is defined by an objective function that defines a linear function to be minimized or maximized, e.g.

$$min\ c^T x$$

where $c$is a vector of coefficients and $x$is the vector of decision variables, whose constrains compose a linear equalities or inequalities, e.g.

A MILP problem is defined as:

$$min\ \ c^T x subject\ to\ Ax \leq b,\ x \in \mathbb{Z}^k \times \mathbb{R}^{n-k}$$

Some decision variables (dominant) are restricted to integer or binary values, while others remain continuous.

This combination allows MILP to capture both **quantitative trade-offs** (continuous variables) and **discrete choices** (integer variables), making it suitable for problems where decisions involve yes/no choices, counts, or assignments.

We define the following sets and parameters:

- F: set of flights, indexed by f.
- R_f: set of candidate routes for flight f.
- c_{fr}: fuel cost of assigning flight $f$to route $r \in R_f$.
- $p_{(f,r),(f',r')}$: conflict penalty if flight $f$on route $r$conflicts with flight f'on route r'.

Decision variables:

- $x_{fr} \in \{0,1\}$: equals 1 if flight $f$is assigned to route r, 0 otherwise.

The MILP formulation is:

$$\min \sum_{f \in F} \sum_{r \in R_f} c_{fr} x_{fr} + \sum_{f \in F} \sum_{r \in R_f} \sum_{f' \in F, f' \neq f} \sum_{r' \in R_{f'}} p_{(f,r),(f',r')} x_{fr} x_{f'r'}$$

Subject to:

$$\sum_{r \in R_f} x_{fr} = 1 \forall f \in F$$

$$x_{fr} \in \{0,1\} \forall f \in F, r \in R_f$$

This ensures each flight is assigned exactly one route while minimizing total fuel and conflict penalties.

6.2 QUBO Formulation

The same problem can be reformulated as a **Quadratic Unconstrained Binary Optimization (QUBO)** problem, suitable for quantum optimization. In fact, QUBO has

emerged as a central paradigm in combinatorial optimization, particularly due to its connection with Ising models and quantum annealing. While Mixed Integer Linear Programming (MILP) remains the classical gold standard for solving optimization problems with discrete and continuous variables, QUBO provides a streamlined binary-only formulation that is well-suited for quantum and neuromorphic computing. This article compares QUBO and MILP, highlighting their mathematical structures, applications, and advantages, with special attention to QUBO's role in quantum computing.

A QUBO problem is defined as:

$$\min_{x\in\{0,1\}^n} x^T Q x$$

where $Q$is a symmetric matrix of coefficients and $x$is a binary vector. Constraints are incorporated as penalty terms directly into Q, making the problem unconstrained but still representative of complex In analogy with MILP, one can define binary variables:

- $x_{fr} \in \{0,1\}$: same as above.

Objective function:

$$\min \sum_{f\in F} \sum_{r\in R_f} c_{fr}x_{fr} + \sum_{f\in F} \sum_{r\in R_f} \sum_{f'\in F, f'\neq f} \sum_{r'\in R_{f'}} p_{(f,r),(f',r')}x_{fr}x_{f'r'} + \lambda \sum_{f\in F} (\sum_{r\in R_f} x_{fr} - 1)^2$$

Here, the first two terms represent fuel costs and conflict penalties, while the third term enforces the constraint that each flight must be assigned exactly one route. The parameter $\lambda$is a large positive constant that penalizes infeasible assignments.

This QUBO formulation can be directly mapped to an Ising Hamiltonian, enabling implementation on quantum hardware via QAOA or quantum annealing. The quadratic penalty term ensures feasibility, while the linear and quadratic cost terms capture operational objectives and conflicts.

6.3. QUBO mapping and QAOA

To apply QAOA, we encode Equation (6) into a binary quadratic objective:

$$Q(x) = \sum_{i,m} w_{i,m}\, x_{i,m} + \sum_{i<j} \sum_{m,n} c_{(i,m),(j,n)}\, x_{i,m}\, x_{j,n}$$

Where:

$w_{i,m} = \text{Fuel}_{i,m} + \text{WeatherPenalty}_{i,m} + \text{WindAdj}_{i,m}$and $c_{(i,m),(j,n)} = \text{ConflictPenalty}_{(i,m),(j,n)}$. The one-route-per-flight constraint in Equation (5) is enforced via penalty terms:

$$Q_{\text{cons}}(x) = \lambda \sum_i (\sum_m x_{i,m} - 1)^2 \qquad \text{(X)}$$

yielding the final QUBO objective

$$Q_{\text{total}}(x) = Q(x) + Q_{\text{cons}}(x) \qquad (9)$$

QAOA constructs a cost Hamiltonian $H_C$with eigenvalues proportional to $Q_{\text{total}}(x)$and alternates between applying $e^{-i\gamma H_C}$and a mixer $e^{-i\beta H_M}$. Classical optimization over (γ, β)biases measurement outcomes toward low-cost bitstrings, which represent globally coherent route selections satisfying constraints and minimizing conflicts.

A. <u>Problem formulation</u>

We model total cost for a set of flights as the sum of fuel-related terms, weather adjustments, and conflict penalties. For flight i selecting route m with distance $d_{i,m}$, cruise speed v, and fuel burn rate r, the baseline fuel model is

$$\text{Fuel}_{i,m} = \frac{d_{i,m}}{v} \cdot r \qquad (1)$$

Weather multipliers and wind adjustments modify this baseline:

$$\text{WeatherPenalty}_{i,m} = \alpha_{i,m} \cdot \text{Fuel}_{i,m}, \alpha_{i,m} \geq 0 \qquad (2)$$

$$\text{WindAdj}_{i,m} = \beta_{i,m} \cdot \text{Fuel}_{i,m}, \beta_{i,m} \in \mathbb{R} \qquad (3)$$

Airspace conflicts add penalties for shared waypoints between selected routes:

$$\text{ConflictPenalty}_{(i,m),(j,n)} = \begin{cases} P & \text{if routes } (i,m) \text{ and } (j,n) \text{ share any waypoint} \\ 0 & \text{otherwise} \end{cases}$$

Typical parameters used in our experiments include $v = 450$knots and $r = 2500$kg/hr (B737 exemplar), thunderstorm multipliers $\alpha \approx 5$, and a conflict penalty $P = 10000$kg for waypoint sharing.

B. <u>Decisions variables and constraints</u>

Let $x_{i,m} \in \{0,1\}$indicate whether flight i selects route m. Each flight selects exactly one route:

$$\sum_{m=1}^{M_i} x_{i,m} = 1, \forall i \in \{1, \dots, N\} \qquad (5)$$

The total cost for a selection $x$is

$$\text{TotalCost}(x) = \sum_{i} \sum_{m} x_{i,m}(\text{Fuel}_{i,m} + \text{WeatherPenalty}_{i,m} + + \sum_{i<j} \sum_{m} \sum_{n} x_{i,m}\, x_{j,n}\, \text{ConflictPenalty}_{(i,m),(j,n)}$$

This can be posed as a Mixed Integer Linear Program by linearizing pairwise terms or as a QUBO by directly embedding the quadratic conflict terms in a binary objective.

C. Classical baselines and their limitations

Greedy selection minimizes per-flight route cost independently and assigns $x_{i,m} = 1$for the cheapest m per i. Its appeal is computational efficiency, but the method does not anticipate cross-flight conflicts, so penalties enter after independent choices are made. Weather-aware A* improves local route scoring by integrating detours and weather multipliers into the route cost, but A* remains a per-flight optimizer. In scenarios where two locally cheap routes share a waypoint, both Greedy and A* converge to the same globally suboptimal selection because they neglect the pairwise coupling terms in Equation (6). This illustrates a general limitation of local methods in globally coupled routing tasks.

6.4. Didactic experiment: two flights, two routes

We reproduce the illustrative case: two flights (F0, F1), two routes each (R0, R1), with base fuel costs and a conflict penalty if both selected routes share waypoint A. Local methods select R0 for both flights due to fuel values (5 kg and 4 kg), but they incur a 10 kg conflict penalty, yielding 19 kg total. QAOA, by optimizing the global selection, increases one local fuel choice to 6 kg (R1) but avoids the 10 kg penalty, resulting in a 10 kg total, a marked improvement that reveals the benefit of globally aware optimization.

Method	Route selection	Fuel (kg)	Conflict penalty (kg)
Greedy	[R0, R0]	9	10
A* only	[R0, R0]	9	10
A* + QAOA	[R1, R0]	10	0

Table 6. Total cost comparison

Greedy and A* both produce 19 kg due to the conflict penalty, while QAOA achieves 10 kg by selecting [R1, R0], illustrating that a slightly higher local fuel choice can be globally optimal when it prevents conflicts.

The contrast between local and global optimization is stark in coupled routing environments. It makes explicit the pairwise terms that local methods ignore. By embedding conflicts directly into the objective and constraints, QAOA searches the combinatorial space for coherent selections that respect one-route-per-flight and minimize conflicts. Although our experiment is didactic, the structure generalizes to realistic airline tail-assignment and multi-flight route planning: large conflict penalties will dominate local fuel savings, and globally aware optimization will consistently prefer conflict-free route sets even when individual routes are not locally minimal.

6.5. Summarization of models

The MILP formulation provides a classical optimization framework that can be solved using branch-and-bound or cutting-plane methods, offering exact solutions but with exponential worst-case complexity. The QUBO formulation, by contrast, is tailored for quantum optimization, allowing exploration of the solution space through quantum superposition and interference.

Basically, the QUBO simplifies the formulation, reducing all variables to binary, avoiding the complexity of mixed domains in MILP. Constraints are embedded as quadratic penalties, eliminating explicit constraint handling. Additionally, the QUBO maps directly to Ising Hamiltonians, making it the native problem class for quantum annealers such as D-Wave systems. MILP requires reformulation before quantum execution.

Additionally, about the scalability QUBO has benefits from heuristic solvers (simulated annealing, tabu search) and hybrid quantum-classical methods that explore large solution spaces efficiently. It means that very complex NP-hard problems can be reformulated into QUBO, enabling a unified approach across domains such as finance, logistics, and machine learning

7. CONCLUSION

The idea in this article shows that integrating weather-aware A* with QAOA transforms aviation route planning from a set of decoupled local choices into a coherent global optimization problem. The QUBO formulation captures fuel, weather, wind, and conflicts in a single cost function, while constraint penalties ensure feasibility. In some cases, QAOA achieves a 47% reduction in total cost by deliberately avoiding a high-penalty conflict. The

implications are clear: when cross-flight interactions are significant, hybrid classical–quantum methods can outperform purely local heuristics.

Future work should address three directions: first, scaling to realistic network sizes with many flights and waypoint graphs while maintaining constraint fidelity; second, ingesting real-time weather and airspace data to update α, β, and conflict maps dynamically; and third, validating performance on quantum hardware and benchmarking against advanced classical global solvers (MILP and metaheuristics). These steps will clarify the regimes in which QAOA offers practical advantages and inform operational deployment.

APPENDICES

Appendix A: Python/Qiskit implementation for the 2×2 experiment

```
python
# IrisPathQ: QUBO + QAOA for 2 flights × 2 routes
# Dependencies: qiskit, numpy

import numpy as np
from qiskit import QuantumCircuit, Aer, execute
from qiskit.opflow import I, Z, PauliSumOp
from qiskit.algorithms import QAOA
from qiskit.utils import algorithm_globals
from qiskit.algorithms.optimizers import COBYLA

algorithm_globals.random_seed = 42

# Decision variables: x0R0, x0R1, x1R0, x1R1 (4 qubits)
# Index mapping
q = {"x0R0": 0, "x0R1": 1, "x1R0": 2, "x1R1": 3}

# Costs (kg)
fuel = {"x0R0": 5, "x0R1": 6, "x1R0": 4, "x1R1": 7}
conflict_pairs = {("x0R0", "x1R0"): 10}  # shared WAYPOINT_A

# Penalty lambda for one-route-per-flight constraints
lam = 20.0

# Helper: binary variable operator in Z-basis
# x = (I - Z)/2
def x_op(idx, num_qubits=4):
    op = (I ^ num_qubits)
    for i in range(num_qubits):
        if i == idx:
            op = op.compose((I ^ (num_qubits - i - 1)) ^ ((I - Z) / 2) ^ (I ^ i))
            break
    return op

# Build cost Hamiltonian H_C ~ Q_total(x)
num_qubits = 4
H = 0 * (I ^ num_qubits)

# Linear terms: sum w_i x_i
for var, w in fuel.items():
    H += w * x_op(q[var], num_qubits)

# Quadratic conflict terms: sum c_ij x_i x_j
for (u, v), c in conflict_pairs.items():
    H += c * x_op(q[u], num_qubits) @ x_op(q[v], num_qubits)

# Constraints: (x0R0 + x0R1 - 1)^2 + (x1R0 + x1R1 - 1)^2
def constraint_square(vars_for_flight):
    s = 0 * (I ^ num_qubits)
    for var in vars_for_flight:
        s += x_op(q[var], num_qubits)
    # s^2 = sum x_i^2 + 2 sum x_i x_j
    # But since x_i^2 = x_i for binary, we can expand directly:
    terms = 0 * (I ^ num_qubits)
    # linear part
    for var in vars_for_flight:
        terms += x_op(q[var], num_qubits)
    # quadratic part
    for i in range(len(vars_for_flight)):
        for j in range(i + 1, len(vars_for_flight)):
            terms += 2 * (x_op(q[vars_for_flight[i]], num_qubits) @ x_op(q[vars_for_flight[j]], num_qubits))
    return terms - 2 * (sum([x_op(q[v], num_qubits) for v in vars_for_flight])) + 1 * (I ^ num_qubits)

H += lam * constraint_square(["x0R0", "x0R1"])
H += lam * constraint_square(["x1R0", "x1R1"])

H_op = PauliSumOp.from_operator(H.to_matrix())

# QAOA setup
optimizer = COBYLA(maxiter=100)
qaoa = QAOA(optimizer=optimizer, reps=2)
result = qaoa.compute_minimum_eigenvalue(operator=H_op)

print("QAOA energy:", result.eigenvalue.real)

# Extract most likely bitstring via sampling from the optimal circuit
qc = qaoa.get_optimal_circuit()
backend = Aer.get_backend('qasm_simulator')
shots = 8192
res = execute(qc.measure_all(inplace=False), backend=backend, shots=shots).result()
counts = res.get_counts()

best_bitstring = max(counts.items(), key=lambda kv: kv[1])[0]
# Bit ordering is q3 q2 q1 q0 in counts; map back to variables
bits = list(map(int, best_bitstring[::-1]))
selection = {var: bits[idx] for var, idx in q.items()}

print("Counts:", counts)
print("Selection:", selection)
# Compute total cost for the selection
def total_cost(sel):
    # Linear cost
    linear = sum(fuel[v] for v in sel if sel[v] == 1)
    # Conflict
    conf = sum(c for (u, v), c in conflict_pairs.items() if sel[u] == 1 and sel[v] == 1)
    return linear + conf

print("Total cost (kg):", total_cost(selection))
```

Appendix B: C-style pseudocode for data, A*, and cost matrix

```
c
// IrisPathQ: Data loading, A* routes, and cost matrix construction (pseudocode)

struct Flight { int id; char origin[8]; char destination[8]; };
struct Waypoint { char name[16]; double lat, lon; };
struct WeatherCell { double lat, lon, radius_nm; double storm_multiplier; double wind_factor; };

Flight flights[N];
Waypoint waypoints[W];
WeatherCell weather_cells[K];

// Load CSVs into arrays: flights.csv, waypoints.csv, weather.csv
void load_data() {
    // parse files, populate arrays
}

double route_distance_nm(Route r) { /* haversine over waypoints */ }
double fuel_cost_kg(double distance_nm, double v_knots, double r_kgph) {
    return (distance_nm / v_knots) * r_kgph;
}

double weather_penalty(Route r) {
    double penalty = 0.0;
    for (segment in r.segments) {
        for (cell in weather_cells) {
            if (segment_intersects_cell(segment, cell)) {
                penalty += cell.storm_multiplier * segment.base_fuel;
            }
        }
    }
    return penalty;
}

double wind_adjustment(Route r) {
    double adj = 0.0;
    for (segment in r.segments) {
        adj += segment.wind_factor * segment.base_fuel;
    }
    return adj;
}

bool share_waypoint(Route a, Route b) {
    for (wp in a.waypoints) for (wq in b.waypoints)
        if (strcmp(wp.name, wq.name) == 0) return true;
```

```
    return false;
}

// A* per flight to generate candidate routes and costs
void generate_routes_for_flight(Flight f, Route routes_out[M]) {
    // graph edges weighted by base fuel + weather + wind
    // run A* from f.origin to f.destination to produce top-M alternatives
}

// Build cost matrix (linear and quadratic terms)
void build_cost_matrix(Route routes[N][M], double linear[N][M], double quadratic[N][M][N][M], double
conflict_penalty) {
    for (i in 0..N-1) for (m in 0..M-1) {
        double d = route_distance_nm(routes[i][m]);
        double fuel = fuel_cost_kg(d, 450.0, 2500.0);
        double wpen = weather_penalty(routes[i][m]);
        double wind = wind_adjustment(routes[i][m]);
        linear[i][m] = fuel + wpen + wind;
    }
    for (i in 0..N-1) for (j in i+1..N-1)
        for (m in 0..M-1) for (n in 0..M-1) {
            quadratic[i][m][j][n] = share_waypoint(routes[i][m], routes[j][n]) ? conflict_penalty : 0.0;
        }
}
```

Appendix C: Reproducible data schema

```
text
flights.csv
- flight_id, origin_icao, destination_icao, aircraft_type

waypoints.csv
- name, lat_deg, lon_deg

weather.csv
- center_lat_deg, center_lon_deg, radius_nm, storm_multiplier, wind_factor
```

BIBLIOGRAPHY

1. Adamouization (2023). *AI Search Flight Route Planner: Implementation of A*, BFS, DFS*. GitHub.
2. Blanco, M., Borndörfer, R., & Maristany de las Casas, P. (2022). *An A* Algorithm for Flight Planning Based on Idealized Vertical Profiles*. OASIcs ATMOS 2022.
3. Blekos, K., et al. (2023). *Review on QAOA and its Variants.* Quantum Reports, 5(1), 1–25.
4. Combarro, E. F., & González-Castillo, S. (2023). *A Practical Guide to Quantum Machine Learning and Quantum Optimization.* Springer.
5. Farhi, E., Goldstone, J., & Gutmann, S. (2014). *A Quantum Approximate Optimization Algorithm.* arXiv:1411.4028.
6. Fuertes, M., Cadarso, L., Vaze, V., Barnhart, C (2020). *The Tail Assognment Problem: A Case Study at Vueling Airlines. Transportation Research Procedia 52 (2021) 445–452*
7. Nan Xu; Xiang Wu; Miaomiao Zhang; Xianying Chang. *Solve the Constrained Shortest Path Problem Using an Improved A* Algorithm* (2024). IEEE Xplore.
8. Kochenberger, G.A., Glover, F., Wang, H. "QUBO: Quadratic Unconstrained Binary Optimization Problem" (pp 1–26) in Handbook of Combinatorial Optimization. 2024, Feb.
9. Lucas, A. (2014). *Ising formulations of many NP problems.* Frontiers in Physics, 2, 5.
10. Nemhauser, G. L., & Wolsey, L. A. Integer and Combinatorial Optimization. Wiley-Interscience, 1988.
11. NordIQuEst. Flight scheduling optimization using QAOA. https://nordiquest.net/application-library/training-material/qas2024/notebooks/flight_scheduling_optimization_qaoa.html
12. Rosenberg, G., et al. (2016). *Solving the Tail Assignment Problem with Quantum Annealing.* IEEE Transactions on Emerging Topics in Computing, 4(2), 211–217.
13. Stollenwerk, T., et al. (2019). *Flight Gate Assignment with Quantum Annealing.* arXiv:1902.04022.
14. Ye, M., Zhao, J., Guan, Q., & Zhang, X. (2024). *Research on eVTOL Air Route Network Planning Based on Improved A* Algorithm*. Sustainability, 16(2), 561.